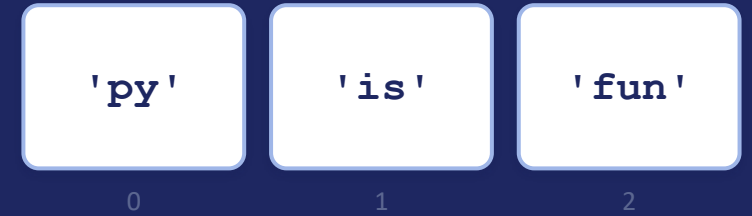


Python Lists

From Fundamentals to Advanced Idioms — Indexing, Methods, Comprehensions & Performance



Learning Objectives

- Create, access, and modify lists using correct Python syntax
- Apply indexing, slicing, and unpacking with confidence
- Select the right built-in method for a given task
- Write list comprehensions to filter and transform data
- Reason about time complexity and avoid common list bugs

By the end of this session

- You will read, write, and debug real list-manipulation code
- You will explain WHY each operation behaves the way it does
- You will be ready for Lab 1 exercises on list operations

Prerequisites

- Comfortable with variables, numbers, and strings
- Can write and read for-loops and while-loops
- Understands if / elif / else conditional logic
- Has written and called basic Python functions
- No prior experience with lists is assumed

Quick self-check

- Can you predict the output of a simple for-loop?
- Do you know the difference between = and ==?
- If yes to both, you are ready for this chapter

Real Programs Manage Collections

RUNNABLE

- A single variable holds one value — real data rarely stops at one
- A list holds an ordered, changeable sequence of items
- Think: student marks, product names, sensor readings
- Lists are Python's default, most flexible collection type

```
marks = [78, 85, 92, 67, 88]

print(marks)
print(type(marks))
```

78

0

85

1

92

2

67

3

88

4

Output

```
[78, 85, 92, 67, 88]
<class 'list'>
```

Edge case An empty list `marks = []` is perfectly valid — length 0, no error.

Lists vs Other Collections

Type	Ordered	Mutable	Duplicates	Example
<code>list</code>	Yes	Yes	Yes	<code>[1, 2, 3]</code>
<code>tuple</code>	Yes	No	Yes	<code>(1, 2, 3)</code>
<code>set</code>	No	Yes	No	<code>{1, 2, 3}</code>
<code>dict</code>	Yes (3.7+)	Yes	Keys unique	<code>{"a": 1}</code>
<code>array</code> (module)	Yes	Yes	Yes, one type	<code>array('i',[1,2])</code>

Creating Lists

RUNNABLE

- List literal: square brackets with comma-separated items
- `list()` constructor converts any iterable — string, tuple, range
- Empty list: `[]` or `list()`
- Lists can mix types, though same-type lists are common practice

```
a = [1, 2, 3]
b = list("abc")
c = list(range(5))
empty = []

print(b, c)
```

Output

```
['a', 'b', 'c'] [0, 1, 2, 3, 4]
```

Edge case `list(5)` raises `TypeError` — an int is not iterable, only sequences convert.

Indexing: Positive and Negative

RUNNABLE

- Indexing starts at 0, not 1
- Negative indices count backward from the end (-1 = last item)
- Indexing returns a single element, not a new list
- An out-of-range index raises IndexError

```
fruits = ["apple", "banana", "cherry"]  
  
print(fruits[0])  
print(fruits[-1])
```

apple

0

-3

banana

1

-2

cherry

2

-1

Output

```
apple  
cherry
```

Edge case fruits[5] -> IndexError: list index out of range (list only has 3 items).

Slicing: start : stop : step

RUNNABLE

- Syntax: `list[start:stop:step]` — stop is always excluded
- Omit start to begin at 0; omit stop to go to the end
- A negative step reverses the direction of the slice
- Slicing always returns a new list, never a single item

```
nums = [10, 20, 30, 40, 50]

print(nums[1:4])
print(nums[:2])
print(nums[::-1])
```

Output

```
[20, 30, 40]
[10, 30, 50]
[50, 40, 30, 20, 10]
```

Edge case `nums[10:20]` returns `[]` — out-of-range slices clamp silently, no error raised.

Nested Lists & Unpacking

RUNNABLE

- Lists can contain lists — useful for grids or matrices
- Chain indices to reach a nested element: `grid[row][col]`
- Unpacking assigns list items straight to variables
- Star (*) unpacking captures any remaining items into a list

```
grid = [[1, 2], [3, 4]]
print(grid[1][0])

a, b, *rest = [1, 2, 3, 4]
print(a, b, rest)
```

Output

```
3
1 2 [3, 4]
```

Edge case `a, b = [1, 2, 3]` -> `ValueError: too many values to unpack.`

Membership, Concatenation, Repetition

RUNNABLE

- `in` / `not in` check whether a value exists — returns `True/False`
- `+` concatenates two lists into a brand-new list
- `*` repeats a list's elements a given number of times
- Both `+` and `*` build new lists; they never mutate the originals

```
a = [1, 2]
b = [3, 4]

print(3 in a)
print(a + b)
print(a * 3)
```

Output

```
False
[1, 2, 3, 4]
[1, 2, 1, 2, 1, 2]
```

Edge case `in` performs a linear scan — checking membership in a huge list repeatedly is slow (see Performance section).

Iteration: enumerate, reversed, zip

RUNNABLE

- for item in list iterates values directly, no index needed
- enumerate(list) pairs each item with its index
- reversed(list) iterates back-to-front without copying data
- zip(list1, list2) pairs items from two lists together

```
names = ["Ann", "Bob"]
scores = [90, 80]

for i, n in enumerate(names):
    print(i, n)
for n, sc in zip(names, scores):
    print(n, sc)
```

Output

```
0 Ann
1 Bob
Ann 90
Bob 80
```

Edge case zip() stops at the shortest list if lengths differ — no error, just truncation.

Mutability & Aliasing

RUNNABLE

- Lists are mutable — their contents can change after creation
- Assigning a list to another name does NOT copy it
- Both names now point to the same underlying list object
- `id()` reveals whether two variables share one object

```
a = [1, 2, 3]
b = a
b.append(4)

print(a)
print(a is b)
```

Output

```
[1, 2, 3, 4]
True
```

Edge case This surprises most learners — a changed even though only `b.append()` was called.

Shallow Copy vs Deep Copy

RUNNABLE

- `b = a` shares the reference — not a copy at all
- `.copy()`, `list(a)`, or `a[:]` make a shallow copy
- Shallow copy: new outer list, but nested objects are still shared
- `copy.deepcopy()` recursively copies every nested object

```
import copy
a = [[1, 2], [3, 4]]
shallow = a.copy()
deep = copy.deepcopy(a)

shallow[0].append(99)
print(a)
print(deep)
```

Output

```
[[1, 2, 99], [3, 4]]
[[1, 2], [3, 4]]
```

Edge case Even a shallow copy protects the OUTER list — `a.append(x)` after copying does not affect `a`.

Methods I: append, extend, insert

RUNNABLE

- `append(x)` adds ONE item to the end — $O(1)$ amortized
- `extend(iterable)` adds every item from another iterable
- `insert(i, x)` inserts at position `i` — $O(n)$, shifts later items
- `append` vs `extend`: single item vs multiple items

```
nums = [1, 2]
nums.append(3)
nums.extend([4, 5])
nums.insert(0, 0)

print(nums)
```

Output

```
[0, 1, 2, 3, 4, 5]
```

Edge case `nums.append([4, 5])` adds ONE nested list, not two items — a very common student bug.

Methods II: remove, pop, clear, index, count

RUNNABLE

- `remove(x)` deletes the FIRST matching value — $O(n)$ scan
- `pop(i)` removes and returns the item at index `i` (default: last)
- `clear()` empties the list in place
- `index(x)` finds a value's position; `count(x)` counts occurrences

```
nums = [1, 2, 3, 2]
nums.remove(2)
last = nums.pop()

print(nums, last)
print(nums.index(3))
```

Output

```
[1, 3] 2
1
```

Edge case `nums.remove(99)` on a missing value raises `ValueError`:
`list.remove(x): x not in list.`

sort() vs sorted() + Custom Keys

RUNNABLE

- `list.sort()` sorts IN PLACE and returns None
- `sorted(list)` returns a NEW sorted list; original is unchanged
- `key=` sorts by the result of a function, not the raw values
- `reverse=True` sorts in descending order

```
words = ["banana", "kiwi", "fig"]
words.sort(key=len)
print(words)

print(sorted([3, 1, 2], reverse=True))
```

Output

```
['fig', 'kiwi', 'banana']
[3, 2, 1]
```

Edge case `result = nums.sort()` sets `result` to `None` — a classic bug from assuming `sort()` returns the list.

Sort Stability, Reverse, Performance

RUNNABLE

- Python's sort is Timsort — stable and adaptive
- Stable: items with equal keys keep their original relative order
- `reverse()` flips a list in place — $O(n)$, no comparisons needed
- Sorting is $O(n \log n)$; avoid re-sorting inside a tight loop

```
data = [("b", 1), ("a", 1), ("c", 0)]
data.sort(key=lambda x: x[1])

print(data)
```

Output

```
[('c', 0), ('b', 1), ('a', 1)]
```

Edge case Notice 'b' still comes before 'a' — both have key 1, and stability preserves their original order.

Built-in Functions on Lists

RUNNABLE

- `len()`, `sum()`, `min()`, `max()` work directly on any list
- `any(list)` is `True` if AT LEAST ONE element is truthy
- `all(list)` is `True` only if EVERY element is truthy
- These functions work on any iterable, not lists alone

```
nums = [4, 0, 9, -2]

print(len(nums), sum(nums), max(nums))
print(any(nums), all(nums))
```

Output

```
4 11 9
True False
```

Edge case `all(nums)` is `False` because 0 is falsy — one zero is enough to fail `all()`.

Flattening Nested Lists

RUNNABLE

- A nested comprehension flattens one level cleanly
- `sum(list_of_lists, [])` is a quick trick for small lists
- `itertools.chain` is more efficient for many/large sublists
- Optimize for readability first, then performance if needed

```
matrix = [[1, 2], [3, 4], [5]]

flat = [x for row in matrix for x in row]
print(flat)
```

Output

```
[1, 2, 3, 4, 5]
```

Edge case `sum(matrix, [])` also works here but silently degrades to $O(n^2)$ on very large inputs.

Chunking & Sliding Windows

RUNNABLE

- Chunking splits a list into fixed-size groups
- A sliding window inspects overlapping subsequences
- Both patterns show up constantly in data-processing pipelines
- List slicing inside a comprehension keeps the code readable

```
data = [1, 2, 3, 4, 5, 6, 7]
size = 3
chunks = [data[i:i+size]
           for i in range(0, len(data), size)]

print(chunks)
```

Output

```
[[1, 2, 3], [4, 5, 6], [7]]
```

Edge case The last chunk may be shorter than 'size' — always design code to handle a partial final chunk.

Comprehensions: The Basics

RUNNABLE

- Syntax: [expression for item in iterable]
- Builds an entire new list in one readable line
- Equivalent to a for-loop with append, but faster and terser
- Works with any iterable — list, range, string, file

```
squares = [n ** 2 for n in range(6)]  
  
print(squares)
```

Output

```
[0, 1, 4, 9, 16, 25]
```

Edge case range(6) is never materialized as a full list first — the comprehension consumes it lazily.

Comprehensions: Conditionals

RUNNABLE

- Trailing if FILTERS which items are included
- if...else BEFORE the expression transforms every item
- Filter form: [x for x in data if condition]
- Transform form: [x if condition else y for x in data]

```
nums = range(10)
evens = [n for n in nums if n % 2 == 0]
labels = ["even" if n % 2 == 0 else "odd"
          for n in nums]

print(evens)
print(labels[:4])
```

Output

```
[0, 2, 4, 6, 8]
['even', 'odd', 'even', 'odd']
```

Edge case Mixing up the two forms is common — filtering if goes at the END, transforming if/else goes at the START.

Nested Comprehensions & Generator Expressions

RUNNABLE

- Nested comprehensions build lists of lists — e.g., grids
- Generator expressions use () instead of [] — lazily evaluated
- A generator computes one value at a time, saving memory
- Prefer a generator when you only need to iterate once

```
grid = [[r*c for c in range(3)] for r in range(3)]
gen = (n ** 2 for n in range(1000000))

print(grid)
print(next(gen))
```

Output

```
[[0, 0, 0], [0, 1, 2], [0, 2, 4]]
0
```

Edge case `sum(gen)` works once; iterating `gen` a second time yields nothing — generators are single-use.

Time Complexity of Common Operations

Operation	Complexity	Why
<code>list[i]</code>	$O(1)$	Direct memory offset calculation
<code>append(x)</code>	$O(1)$ amortized	Occasional resize doubles capacity
<code>insert(0, x)</code>	$O(n)$	Shifts every existing element right
<code>pop()</code> (end)	$O(1)$	No shifting of remaining elements
<code>pop(0)</code>	$O(n)$	Shifts every remaining element left
<code>x in list</code>	$O(n)$	Linear scan until a match is found
<code>sort()</code>	$O(n \log n)$	Timsort, a comparison-based sort

When to Reach for deque, array, tuple, numpy

RUNNABLE

- collections.deque: $O(1)$ appends/pops from BOTH ends
- array module: compact storage for one numeric type
- tuple: immutable and hashable — safe as a dict key
- numpy array: vectorized math over large numeric datasets

```
from collections import deque

q = deque([1, 2, 3])
q.appendleft(0)
print(q)
```

Output

```
deque([0, 1, 2, 3])
```

Edge case Optional import — pure-Python fallback is `list.insert(0, x)`, just $O(n)$ instead of $O(1)$.

Iterating While Modifying, Mutable Defaults

RUNNABLE

- Removing items while iterating silently SKIPS elements
- Iterate over a copy — `list[:]` — when removing conditionally
- `def f(x, items=[])` reuses the SAME list across every call
- Use `None` as the default; create the list inside the function

```
def add_item(x, items=None):  
    if items is None:  
        items = []  
    items.append(x)  
    return items  
  
print(add_item(1), add_item(2))
```

Output

```
[1] [2]
```

Edge case With `items=[]` as the default instead, the second call would return `[1, 2]` — the bug is state leaking between calls.

Copy Semantics & Expensive Concatenation

RUNNABLE

- `a = b` copies the REFERENCE, never the list's contents
- Slicing always returns a copy — Python lists have no 'view'
- `result = result + [x]` inside a loop is $O(n)$ per call
- Prefer `list.append()` in a loop, or a comprehension instead

```
# Slow:  $O(n^2)$  overall
result = []
for x in range(1000):
    result = result + [x]

# Fast:  $O(n)$  overall
result = [x for x in range(1000)]
```

Output

Both produce the same list —
only the performance differs.

Edge case The slow version rebuilds a brand-new list on every single iteration, copying everything each time.

Stack & Queue with Lists

RUNNABLE

- A list works naturally as a STACK: append pushes, pop pops
- `append()/pop()` at the end are both $O(1)$ — ideal for a stack
- For a QUEUE (FIFO), use `collections.deque` instead
- `pop(0)` on a plain list is $O(n)$ — a poor choice for queues

```
stack = []
stack.append(1)
stack.append(2)

print(stack.pop())
print(stack)
```

Output

```
2
[1]
```

Edge case LIFO means Last In, First Out — the most recently pushed item is always removed first.

Top-K Values & Histogram

RUNNABLE

- `sorted(data)[-k:]` gives the k largest values
- Counting occurrences builds a simple frequency histogram
- `collections.Counter` automates frequency counting entirely
- Pick the tool that matches your data size and clarity needs

```
from collections import Counter

votes = ["A", "B", "A", "C", "A", "B"]
print(Counter(votes).most_common(2))
```

Output

```
[('A', 3), ('B', 2)]
```

Edge case `most_common()` ties break by first-encountered order, thanks to dict's insertion-order guarantee.

Remove Duplicates, Preserve Order

RUNNABLE

- `set(list)` removes duplicates but LOSES the original order
- `dict.fromkeys(list)` removes duplicates and keeps first-seen order
- A manual loop with a 'seen' set also preserves order
- `dict.fromkeys()` is usually the clearest, fastest idiom

```
items = [3, 1, 3, 2, 1]

unique = list(dict.fromkeys(items))
print(unique)
```

Output

```
[3, 1, 2]
```

Edge case `list(set(items))` would give the same values but in an unpredictable order — a subtle real-world bug.

Converting Lists: CSV & JSON

RUNNABLE

- `list(str) / "".join(list)` convert between text and lists
- `json.dumps()/json.loads()` convert lists to and from JSON text
- `csv.reader` yields each row as a list of strings
- CSV values always arrive as strings — convert types explicitly

```
import json

data = [1, 2, 3]
text = json.dumps(data)
back = json.loads(text)

print(text, back)
```

Output

```
[1, 2, 3] [1, 2, 3]
```

Edge case `json.loads('[1, "2", 3]')` keeps "2" as a string — JSON preserves each value's own type.

Testing List-Manipulation Code

RUNNABLE

- `assert list == expected` catches bugs early during development
- doctest examples embed runnable tests inside docstrings
- unittest / pytest formalize testing for larger projects
- Always test edge cases: empty list, one item, duplicates

```
def double(items):  
    """  
    >>> double([1, 2, 3])  
    [2, 4, 6]  
    """  
    return [x * 2 for x in items]  
  
assert double([]) == []
```

Output

No output means all assertions passed.

Edge case `double([])` must return `[]` — forgetting to test the empty-list case is a common source of production bugs.

Cheat Sheet

Create & Access

- `[1, 2, 3]`
- `list(iterable)`
- `lst[i]` `lst[-1]`
- `lst[a:b:c]`
- `a, b, *r = lst`
- `grid[row][col]`

Methods (mutate)

- `append(x)`
`extend(it)`
- `insert(i, x)`
- `remove(x)` `pop(i)`
- `clear()` `sort()`
- `reverse()`
- `index(x)` `count(x)`

Non-mutating & Copy

- `sorted(lst)`
`reversed(lst)`
- `lst + other` `lst * n`
- `lst.copy()` `lst[:]`
- `copy.deepcopy(lst)`
- `len` `sum` `min` `max`
- `any(lst)` `all(lst)`

Comprehensions

- `[x for x in it]`
- `[x for x in it if c]`
- `[x if c else y for x in it]`
- `[x for row in m for x in row]`
- `(x for x in it)` # generator

Quiz — 10 Questions

Questions 1–5

- 1. What does `fruits[-1]` return?
- 2. True/False: `sorted()` modifies the original list.
- 3. Which method adds a single item to the end?
- 4. What is the output of `[1,2] + [3,4]`?
- 5. Fill in the blank: ____ removes and returns the last item.

Questions 6–10

- 6. What is wrong with `def f(x, items=[])`?
- 7. What is the time complexity of `pop(0)`?
- 8. Write a comprehension for squares of even 0–10.
- 9. What does `list(dict.fromkeys(items))` do?
- 10. Give a one-line difference: shallow vs deep copy.

5 Progressive Exercises

1–3: Basic → Intermediate

- 1. Find max & min of a list WITHOUT using `max()/min()`.
- 2. Remove all negative numbers using a comprehension.
- 3. Flatten a 2-level nested list into one flat list.

4–5: Intermediate → Advanced

- 4. Write `unique_preserve_order(lst)` — no duplicates, order kept.
- 5. Track only the 3 most recently added items in a list.
- Test each with: empty list, one item, and duplicate values.

Summary & References

Key Takeaways

- Lists are ordered, mutable sequences — Python's default collection
- Indexing, slicing, and unpacking are the core access patterns
- Methods mutate in place; `sorted()`/`copy()` build new lists
- Comprehensions replace most manual for-loops, cleanly
- Know the Big-O of an operation before using it in a hot loop

References & Further Reading

- Python Docs — Data Structures: docs.python.org/3/tutorial/datastructures.html
- Python Docs — collections: docs.python.org/3/library/collections.html
- Real Python — Lists and Tuples in Python
- Real Python — List Comprehensions: A Beginner Guide
- TimeComplexity wiki — wiki.python.org/moin/TimeComplexity